

Лабораторная работа № 1

Демонстрация и защита

Go / Gin / PostgreSQL / Docker / GitHub Actions / Railway

Практическая памятка - подготовлена 3 октября 2026 г.

1. Подготовка перед защитой

Откройте PowerShell в каталоге проекта. Нужны Go согласно go.mod (1.25.6 или более новая совместимая версия), Node.js и Docker Desktop. Docker нужен для локальной демонстрации. Выполняйте команды в одном терминале, чтобы сохранялись переменные.

Вариант А: опубликованный API

```
$baseUrl = 'https://lab1-template-production-6bea.up.railway.app'  
curl.exe -i "$baseUrl/api/v1/persons"
```

Ожидается HTTP 200 и JSON-массив. [] означает отсутствие записей; непустой список также допустим.

Вариант В: локальный Docker Compose

```
docker compose up -d --build  
docker compose ps  
$baseUrl = 'http://localhost:8080'  
curl.exe -i "$baseUrl/api/v1/persons"
```

Ожидается работающий app и postgres со статусом healthy. Освободите порт 8080 перед запуском Compose. Выберите один baseUrl и используйте его далее.

Порядок защиты: 10-15 минут

1. Архитектура и база данных (2 мин).
2. Пять операций CRUD и ошибки (4 мин).
3. Unit-тесты и Mock Service (2 мин).
4. Newman, CI и deployment на Railway (3 мин).
5. Вопросы и оставшиеся требования задания (2 мин).

2. Демонстрация CRUD: создание и чтение

Создайте новую демонстрационную запись. Не предполагается ID = 1: адрес записи берётся из заголовка Location.

POST /api/v1/persons

```
$body = @{
    name = 'Ahmed Demo'
    age = 35
    address = 'Moscow'
    work = 'Developer'
} | ConvertTo-Json

$created = Invoke-WebRequest -UseBasicParsing -Method Post `
    -Uri "$baseUrl/api/v1/persons" `
    -ContentType 'application/json' -Body $body

$created.StatusCode
$created.Headers['Location']
$created.Content.Length
$personUrl = $baseUrl + $created.Headers['Location']
```

Ожидаемые результаты: 201; /api/v1/persons/<реальный ID>; 0. Тело ответа POST пустое, а Location указывает адрес созданного ресурса.

GET созданного человека

```
curl.exe -i "$personUrl"
```

Ожидается 200, application/json и те же name, age, address и work, что в POST. В ответе также есть созданный id.

GET всех людей

```
curl.exe -i "$baseUrl/api/v1/persons"
```

Ожидается 200 и JSON-массив с созданной записью. Данные доступны через оба endpoint для чтения.

Что объяснить

Handler проверяет JSON и передаёт данные в Service. Repository выполняет параметризованный INSERT в PostgreSQL и возвращает ID. Handler формирует заголовок Location.

Если Invoke-WebRequest сообщает об ошибке, проверьте HTTP-статус и логи приложения до продолжения.

3. Демонстрация CRUD: изменение, удаление, ошибки

PATCH: изменение выбранных полей

```
$body = @{ name = 'Ahmed Updated'; address = 'Kazan' } |  
    ConvertTo-Json  
$updated = Invoke-WebRequest -UseBasicParsing -Method Patch `   
    -Uri $personUrl -ContentType 'application/json' -Body $body  
$updated.StatusCode  
$updated.Content  
curl.exe -i "$personUrl"
```

Ожидается 200. Имя изменяется на Ahmed Updated, адрес - на Kazan. Возраст остаётся 35, работа - Developer. Последующий GET подтверждает сохранение изменений.

Необязательные поля запроса представлены указателями. Пропущенные age/address/work равны nil; SQL COALESCE сохраняет прежние значения. Явно переданный возраст 0 учитывается. В этой реализации PATCH требует непустое name.

DELETE и проверка отсутствия записи

```
curl.exe -i -X DELETE "$personUrl"  
curl.exe -i "$personUrl"
```

Ожидается DELETE со статусом 204 и пустым телом. GET после удаления возвращает 404. Удаляйте только созданную демонстрационную запись.

Неверный ID и отсутствие name

```
curl.exe -i "$baseUrl/api/v1/persons/abc"  
Invoke-WebRequest -UseBasicParsing -Method Post `   
    -Uri "$baseUrl/api/v1/persons" `   
    -ContentType 'application/json' -Body '{}'
```

Оба запроса должны вернуть 400. Invoke-WebRequest обычно выдаёт исключение при HTTP-ошибке; здесь это ожидаемо. curl.exe -i показывает ответ вместе со статусом.

Статус	Значение
200	Успешный GET / PATCH
201	Создано; заголовок Location; пустое тело
204	Удалено; пустое тело
400	Неверный ID или JSON запроса
404	Человек не найден
500	Непредвиденная ошибка сервиса или БД

4. Unit-тесты и интеграционные тесты

Запуск и подсчёт unit-тестов

```
go test ./...
go test -v ./internal/handler
go test ./internal/handler -list '^Test'
(go test ./internal/handler -list '^Test' |
  Select-String '^Test').Count
```

Ожидается PASS / ok, пять основных тестовых функций и 25 подтестов. [no test files] для других пакетов - нормально: тесты находятся в internal/handler/person_test.go.

Функция	Подтесты
TestCreatePerson	6
TestGetPersonByID	4
TestGetAllPersons	3
TestUpdatePerson	8
TestDeletePerson	4

Объяснение Mock

Путь unit-теста: HTTP-запрос → Gin router → Handler → Mock Service → recorder. Mock реализует тот же интерфейс, что и настоящий Service. `httptest.NewRequest` создаёт запрос, `httptest.NewRecorder` сохраняет ответ, `ServeHTTP` запускает Handler без сетевого сервера.

Для POST покажите проверки 201, заголовка Location и пустого тела. Mock также проверяет данные, переданные в Service. Эти тесты не проверяют настоящий SQL или PostgreSQL.

Запуск Newman против Railway

```
npx.cmd --yes --package=newman@6 newman run `
  "postman/[inst] Lab1.postman_collection.json" `
  -e "postman/[inst][railway] Lab1.postman_environment.json"
```

Ожидается пять запросов, пять именованных проверок, ноль ошибок. Newman проверяет опубликованный API и настоящую БД. Коллекция создаёт запись и удаляет её в конце. Она не охватывает все возможные ошибочные ситуации.

5. CI, deployment и диагностика

Покажите GitHub Actions

Откройте Repository → Actions → CI. Для push или pull request в master задача test загружает зависимости, запускает unit-тесты, собирает Go-приложение и Docker-образ.

После успешного deployment в upbeat-delight / production запускается Newman tests on Railway. Задача проверяет доступность API, выполняет коллекцию и сохраняет railway-newman-results (JUnit XML). Actions → CI → Run workflow запускает интеграционную проверку текущего deployment вручную.

Skipped не означает ошибку: сборка пропускается для deployment_status, интеграционная задача - для push/PR и неподходящих deployment-событий. Найдите успешный запуск Newman для production.

Покажите Railway и конфигурацию

Покажите сервисы приложения и Postgres, deployment и публичный домен. Приложение читает DATABASE_URL и PORT. В Compose host БД - postgres; на Railway используется reference на сервис Postgres. Не показывайте пароли.

Name: DATABASE_URL
Value: \${Postgres.DATABASE_URL}

Это два отдельных поля переменной. Имя переменной не Postgres; в значение не добавляется DATABASE_URL=. До обработки запросов приложение создаёт persons через CREATE TABLE IF NOT EXISTS.

Краткая диагностика

Симптом	Проверка
502	Deploy logs; запуск БД; Target Port домена должен совпадать с PORT.
localhost:5432 refused	DATABASE_URL отсутствует, пустой или локальный. Примените переменную и повторите deployment.
Порт 8080 занят	Остановите прежнее локальное приложение перед Compose.
Ошибка Classroom	В исходном classroom.yml пустой Heroku baseUrl; это отдельный workflow.

Автооценка Classroom и запись в Google-таблицу ещё требуют настроек преподавателя и не проверены. Зелёный CI не означает успех всех checks репозитория.

6. Вопросы на защите и итоговая проверка

Зачем нужны слои?

Handler отвечает за HTTP, Service выделяет операции приложения, Repository отвечает за SQL. Интерфейс позволяет заменить Service в тестах.

Почему localhost не работает в Docker?

Это адрес самого контейнера приложения. Другой контейнер доступен по имени сервиса; Railway использует переменную подключения БД.

Чем unit-тесты отличаются от Newman?

Unit-тесты изолируют Handler с помощью mock. Newman обращается к опубликованному API и настоящей БД.

Что такое multi-stage build?

Этап Go собирает бинарный файл с выключенным CGO. Этап Alpine содержит бинарный файл без компилятора Go.

Что такое CI и deployment?

CI проверяет тесты и сборку. Railway отдельно собирает и запускает приложение после обновления GitHub. Событие после deployment запускает Newman.

Соответствие исходному заданию

Исходное задание требует Heroku и Docker-deployment через GitHub Actions. Здесь используется Railway с автодеплоем из GitHub. Уточните у преподавателя допустимость замены платформы и способа deployment. Исходный workflow Classroom также не настроен полностью. До решения этих вопросов не заявляйте полное соответствие заданию.

Завершение демонстрации

Проверьте: показаны статусы CRUD; демонстрационная запись удалена; unit-тесты успешны; Newman без ошибок; объяснены CI и запуск после deployment; названы оставшиеся требования.

```
docker compose down
```

Команда останавливает локальные контейнеры и сохраняет том БД. Не используйте down -v, если данные нужно сохранить.